győri szakképzési centrum

Jedlik Ányos
Gépipari és Informatikai
Technikum és Kollégium

9021 Győr, Szent István út 7.
+36 (96) 529-480
+36 (96) 529-448

OM: 203037/003
jedlik@jedlik.eu
www.jedlik.eu

Záródolgozat feladatkiírás

Tanuló(k) neve:	Ress Dominik Ferenc, Varga Vendel
Képzés:	nappali
Szak:	5 0613 12 03 Szoftverfejlesztő és -tesztelő technikus

A záródolgozat címe:

FleetFlow - flottakezelő rendszer

Konzulens:	Bólya Gábor
Beadási határidő:	2026. 04. 15.

Győr, 2026. április 15.

Módos Gábor
igazgató

győri szakképzési centrum

Jedlik Ányos
Gépipari és Informatikai
Technikum és Kollégium

9021 Győr, Szent István út 7.
+36 (96) 529-480
+36 (96) 529-448

OM: 203037/003
jedlik@jedlik.eu
www.jedlik.eu

Konzultációs lap

	A konzultáció		Konzulens aláírása
	ideje	témája	
1.	2026.02.15.	Témaválasztás és specifikáció	
2.	2026.03.14.	Záródolgozat készültségi fokának értékélése	
3.	2026.04.15.	Dokumentáció véglegesítése	

Tulajdonosi nyilatkozat

Ez a dolgozat a saját munkánk eredménye. Dolgozatunk azon részeit, melyeket más szerzők munkájából vettünk át, egyértelműen megjelöltük.
Ha kiderülne, hogy ez a nyilatkozat valótlan, tudomásul vesszük, hogy a szakmai vizsgabizottság a szakmai vizsgáról kizár minket és szakmai vizsgát csak új záródolgozat készítése után tehetünk.

Győr, 2026. április 15.

tanuló aláírása

tanuló aláírása

Tartalomjegyzék

1. Bevezetés 6
1.1 Probléma és megoldás 6
1.2 Kezdet 6
1.2.1 Technológiák 6
1.3 Jövőbeni tervek 7
2. Csapatmunka megvalósítása 8
2.1 Feladatok felosztása 8
2.2 Git és GitHub 8
3. Adatbázis 9
3.1 Adatbázis-kezelő rendszer 9
3.2 Adatbázis-séma áttekintése 9
3.3 Táblák részletes leírása 9
3.3.1 users tábla 9
3.3.2 drivers tábla 10
3.3.3 vehicles tábla 10
3.3.4 vehicle_assignments tábla 11
3.3.5 files tábla 11
3.3.6 fuel_logs tábla 11
3.3.7 trips tábla 12
3.3.8 service_requests tábla 12
3.3.9 calendar_events tábla 13
3.3.10 notifications tábla 13
3.3.11 password_tokens tábla 13
3.4 Adatbázis kapcsolatok 13
4. Backend 14
4.1 Technológia 14
4.2 Projekstruktúra 14
4.3 Hitelesítés és szerepkörök 14
4.4 Jelszótitkosítás 14
4.5 E-mail funkciók 15
4.6 API végpontok 15
4.6.1 Hitelesítés (/api) 15
4.6.2 Sofőrkezelés (/api/admin/drivers) – ADMIN 15
4.6.3 Járműkezelés (/api/admin/vehicles) – ADMIN 15
4.6.4 Jármű–Sofőr hozzárendelés (/api/admin) 16
4.6.5 Útnyilvántartás (/api/trips) 16
4.6.6 Üzemanyag-napló (/api/fuellogs) 16
4.6.7 Szervizigények (/api/service-requests) 16
4.6.8 Naptár (/api/calendarevents) 17
4.6.9 Értesítések (/api/notifications) 17

4.6.10 Profil (/api/profile) ... 17
4.6.11 Statisztikák (/api/statistics) ... 17
4.6.12 Fájlkezelés (/api/files) ... 18
4.7 Lapozás és szűrés ... 18
5. Biztonság és rendszerszintű megoldások ... 19
5.1 JWT alapú hitelesítés részletei ... 19
5.2 Middleware réteg ... 19
5.3 CORS konfiguráció ... 19
5.4 Adatvalidáció ... 19
5.5 Soft delete mechanizmus ... 20
5.6 Fájlfeltöltés és -kezelés ... 20
6. Backend tesztelés ... 21
6.1 Tesztelési megközelítés ... 21
6.2 A tesztprojekt struktúrája ... 21
6.3 Tesztelési segédosztályok ... 22
6.3.1 TestDbContextFactory ... 22
6.3.2 TestControllerFactory ... 22
6.4 Unit tesztek ... 22
6.4.1 OwnDataControllerTests ... 22
6.4.2 EmailServiceTests ... 22
6.5 API integrációs tesztek (Fetch Client kollekció) ... 23
6.6 CI/CD integrált tesztelés ... 23
7. Konténerizáció és publikálás ... 24
7.1 Docker ... 24
7.2 Kubernetes ... 24
7.3 CI/CD Pipeline ... 24
7.3.1 .NET CI (dotnet.yml) ... 25
7.3.2 Docker Build és Deploy (deploy.yml) ... 25
7.3.3 Android Build (android-build.yml – Mobil) ... 25
7.4 Fejlesztői futtatási kézikönyv ... 25
7.5 Környezeti változók ... 26
7.6 GitHub Actions Secrets ... 26
8. Mobil alkalmazás ... 27
8.1 Technológia és architektúra ... 27
8.2 Architektúra – MVVM ... 27
8.3 Hitelesítés és munkamenet-kezelés ... 28
8.4 Alkalmazás képernyői ... 28
8.4.1 Bejelentkezés képernyő ... 28
8.4.2 Irányítópult (Dashboard) ... 28
8.4.3 Utak képernyő (Trip Log) ... 28
8.4.4 Tankolás képernyő (Fuel Log) ... 28
8.4.5 Szerviz képernyő (Service Requests) ... 28

8.4.6 Értesítések képernyő 29
8.4.7 Profil képernyő 29
8.5 Navigáció 29
8.6 Felugró ablakok (Popups) 29
8.6.1 Elfelejtett jelszó felugró (ForgotPasswordPopup) 29
8.6.2 Szerviz részletek felugró (ServiceDetailsPopup) 29
8.6.3 Naptár nap felugró (CalendarDayPopup) 30
8.7 Adatmodellek 30
8.8 Téma támogatás 31
8.9 CI/CD és kiadások 31
9. Webes frontend 32
9.1 Technológia 32
9.2 Projektstruktúra 32
9.3 Hitelesítés és AuthContext 33
9.4 Oldalak és funkciók 33
9.4.1 Bejelentkezés oldal 33
9.4.2 Admin irányítópult (Dashboard) 33
9.4.3 Járműkezelés 33
9.4.4 Sofőrkezelés 33
9.4.5 Útnyilvántartás 33
9.4.6 Üzemanyag-naplók 33
9.4.7 Szervizigények kezelése 34
9.4.8 Sofőr irányítópult 34
9.4.9 Sofőr saját oldalak 34
9.4.10 Értesítések 34
9.5 Reszponzivitás 34
9.6 Webes tesztelés – Cypress E2E 34
9.7 Docker és Kubernetes 35
9.8 CI/CD pipeline 35
9.9 Fejlesztői futtatási kézikönyv 36
10. Források / Irodalomjegyzék 37

1. Bevezetés

1.1 Probléma és megoldás

A mai modern vállalatok életében a gépjárműflotta hatékony kezelése kritikus fontosságú, mégis sok cégnél ez a folyamat papíralapú nyilvántartással, szétszórt táblázatokkal vagy elavult rendszerekkel történik. Az üzemanyag-felhasználás, az útnyilvántartás, a szervizigények és a sofőrök adminisztrációja nehezen áttekinthető, ami felesleges időráfordítást, hibákat és anyagi veszteséget okoz.

A FleetFlow egy modern, webalapú és mobilon is elérhető flottakezelő rendszer, amely megoldást kínál ezekre a problémákra. Az alkalmazás lehetővé teszi a flottaadminisztrátorok számára, hogy egyetlen felületről kezeljék a járműveket, a sofőröket és a hozzárendeléseket, míg a sofőrök mobilalkalmazáson keresztül rögzíthetik útjaikat, tankolásaikat és szervizigényeiket.

1.2 Kezdet

A FleetFlow ötlete abból a felismerésből született, hogy a piacon elérhető flottakezelő megoldások vagy túlságosan drágák és bonyolultak, vagy nem nyújtanak mobil megoldást a sofőrök számára. A projekt célja egy modern, skálázható, könnyen használható rendszer elkészítése volt, amely egy 2 fős fejlesztői csapat záródolgozataként valósult meg.

1.2.1 Technológiák

A projekt technológiai választásainál a modern, iparban is elterjedt megoldásokat részesítettük előnyben, amelyek egyrészt megfelelnek a szakmai elvárásoknak, másrészt lehetőséget adnak a mélyebb ismeretek megszerzésére.

Komponens	Technológia
Backend keretrendszer	ASP.NET Core 10 (C#)
Adatbázis	MySQL 8
ORM	Entity Framework Core 10
Hitelesítés	JWT Bearer token
Jelszóhash	BCrypt.Net
E-mail küldés	SMTP (Gmail)
Fájlkezelés	SixLabors.ImageSharp
API dokumentáció	Swagger / OpenAPI
Mobil alkalmazás	.NET MAUI (C#, XAML)
MVVM könyvtár (mobil)	CommunityToolkit.Mvvm 8.4.0
Weboldal	React 19
Konténerizáció	Docker
Orkesztráció	Kubernetes (Nginx Ingress)
Container Registry	GitHub Container Registry (GHCR)
CI/CD	GitHub Actions
Verziókövetés	Git, GitHub
Kód szerkesztő	Visual Studio / Visual Studio Code

1.3 Jövőbeni tervek

A projekt elkészülte után számos fejlesztési lehetőség merült fel, amelyeket a jövőben tervezünk megvalósítani:

- GPS-alapú valós idejű útkövetés integrálása a mobil alkalmazásba.
- Push értesítések küldése mobilra (pl. lejáró jogosítvány, szerviz határidő figyelmeztetés).
- Többnyelvű (i18n) támogatás hozzáadása a mobil alkalmazáshoz.
- Üzemanyag-hatékonysági elemzések és CO2-kibocsátás kalkulátor.
- Több vállalat kezelése (multi-tenant architektúra) egyetlen rendszerből.

2. Csapatmunka megvalósítása

2.1 Feladatok felosztása

A projekt 2 fős csapat munkájaként valósult meg. Az alapelv az volt, hogy mindkét fejlesztő megismerkedjen a rendszer minden rétegével, miközben a munkák felosztása hatékony párhuzamos fejlesztést biztosított.

Fejlesztési terület	Felelős	Megjegyzés
Backend (ASP.NET Core)	Ress Dominik	Fő felelős: API végpontok, adatbázis, autentikáció, CI/CD
Mobil alkalmazás (.NET MAUI)	Ress Dominik	Fő felelős: MVVM architektúra, összes képernyő
Adatbázis tervezés	Közös	SQL séma, kapcsolatok megtervezése
Webes frontend (React)	Varga Vendel	Fő felelős: összes képernyő
Docker / Kubernetes deploy	Közös	Deployment konfigurációk, GitHub Actions
Tesztelés	Közös	Backend unit tesztek, API tesztek, Cypress tesztek

2.2 Git és GitHub

A verziókövetéshez és a csapatmunka koordinálásához a Git verziókezelő rendszert és a GitHub webes platformot használtuk. Mindhárom projekt (backend, mobil, frontend) külön repositoryban kapott helyet a FleetFlow-Zarodolgozat GitHub szervezeten belül.

- Backend repository: https://github.com/FleetFlow-Zarodolgozat/FleetFlow_backend
- Mobil repository: https://github.com/FleetFlow-Zarodolgozat/FleetFlow_mobil
- Frontend repository: https://github.com/FleetFlow-Zarodolgozat/FleetFlow_frontend

A fejlesztés során feature branch-eket alkalmaztunk; minden nagyobb funkciónak saját branch-e volt, amelyet kód átvizsgálás (code review) után merge-öltünk a main ágba. A commit üzenetek tömören, angolul írják le az elvégzett változtatást.

A GitHub Actions CI/CD pipeline automatikusan futtatja a teszteket és buildeli a Docker image-t minden main branch-re kerülő push után, ezzel biztosítva, hogy a kódbázis mindig stabil és deployolható állapotban legyen.

3. Adatbázis

3.1 Adatbázis-kezelő rendszer

Adatbázis-kezelő rendszernek MySQL 8-at választottunk. A MySQL egy nyílt forráskódú, SQL alapú, többfelhasználós relációs adatbázis-kezelő rendszer, amely rendkívül elterjedt a webes alkalmazások körében. Az adatbázis sémáját a create_tables.sql fájlban definiáltuk, amelyből az adatbázis egyetlen paranccsal inicializálható.

Az ORM (Object-Relational Mapping) réteghez Entity Framework Core 10-et alkalmaztunk, amely lehetővé teszi, hogy C# osztályok formájában kezeljük az adatbázis entitásokat, és LINQ lekérdezésekkel kommunikáljunk az adatbázissal anélkül, hogy nyers SQL-t kellene írni.

Az adatbázis neve: flottakezelo_db. A karakterkódolás utf8mb4, amely teljes Unicode támogatást nyújt, beleértve az ékezetes karaktereket és az emoji-kat is.

3.2 Adatbázis-séma áttekintése

Az adatbázis 11 táblából áll, amelyek a flottakezelés összes üzleti folyamatát lefedik. Az alábbi táblázat összefoglalja a főbb táblákat és azok szerepét:

Tábla neve	Leírás
users	Felhasználók alaptáblája (ADMIN és DRIVER szerepkörrel)
drivers	Sofőr-specifikus adatok, jogosítványadatok (1:1 kapcsolat a users táblával)
vehicles	Járművek adatai (rendszám, VIN, állapot, kilométeróra)
vehicle_assignments	Jármű–sofőr hozzárendelések időszakos nyilvántartása
files	Feltöltött fájlok metaadatai (profilképek, számlák, bizonylatok)
fuel_logs	Üzemanyag-naplók (tankolási bejegyzések sofőrönként)
trips	Útnyilvántartás (megtett utak adatai)
service_requests	Szervizigények teljes életciklussal (REQUESTED → CLOSED)
calendar_events	Naptáresemények (személyes feladatok, szerviz időpontok)
notifications	Rendszerértesítések felhasználóknak
password_tokens	Jelszó-visszaállítási tokenek tárolása

3.3 Táblák részletes leírása

3.3.1 users tábla

A felhasználók alaptáblája. Minden rendszerfelhasználó (adminisztrátor és sofőr egyaránt) ebben a táblában kerül nyilvántartásra.

Oszlop neve	Típus	Leírás
id	BIGINT UNSIGNED PK AI	Egyedi azonosító, elsődleges kulcs
email	VARCHAR(255) UNIQUE NOT NULL	Email cím (bejelentkezési azonosító, egyedi)
password_hash	VARCHAR(255)	BCrypt-tel hashelt jelszó
role	ENUM('ADMIN','DRIVER') NOT NULL	Felhasználói szerepkör
full_name	VARCHAR(255) NOT NULL	Teljes név
phone	VARCHAR(50)	Telefonszám (opcionális)
profile_img_file_id	BIGINT UNSIGNED FK	Profilkép hivatkozása (files tábla)
is_active	TINYINT(1) DEFAULT 1	Aktív-e a fiók (soft deaktiváláshoz)
created_at	DATETIME	Létrehozás időbélyege
updated_at	DATETIME ON UPDATE	Utolsó módosítás időbélyege

3.3.2 drivers tábla

A sofőr-specifikus adatokat tárolja. 1:1 kapcsolatban áll a users táblával: minden DRIVER szerepkörű felhasználóhoz pontosan egy drivers bejegyzés tartozik.

Oszlop neve	Típus	Leírás
id	BIGINT UNSIGNED PK AI	Egyedi azonosító
user_id	BIGINT UNSIGNED FK UNIQUE	Kapcsolódó felhasználó (users.id), CASCADE törlés
license_number	VARCHAR(100) NOT NULL	Jogosítvány száma
license_expiry_date	DATE	Jogosítvány lejárati dátuma
notes	TEXT	Egyéb megjegyzések a sofőrről (opcionális)

3.3.3 vehicles tábla

A flotta járműveit tárolja. Minden jármű egyedi rendszámmal és VIN-nel rendelkezik, amelyek egyediségét adatbázis szintű UNIQUE constraint biztosítja.

Oszlop neve	Típus	Leírás
id	BIGINT UNSIGNED PK AI	Egyedi azonosító
license_plate	VARCHAR(32) UNIQUE NOT NULL	Rendszám (egyedi)
brand	VARCHAR(80) NOT NULL	Gyártmány (márka)
model	VARCHAR(80) NOT NULL	Modell neve
year	INT	Gyártási év
vin	VARCHAR(64) UNIQUE	Alvázszám (egyedi, opcionális)
current_mileage_km	INT DEFAULT 0	Aktuális kilométeróra állás
status	ENUM('ACTIVE','MAINTENANCE','RETIRED')	Jármű állapota
created_at / updated_at	DATETIME	Létrehozás és módosítás időbélyegei

3.3.4 vehicle_assignments tábla

A jármű–sofőr hozzárendelések előzményeit tárolja időszakos formában. Ha az assigned_to mező NULL, az aktív hozzárendelést jelent. A tábla lehetővé teszi, hogy lekérdezhetők legyenek az egyes járművek vagy sofőrök teljes hozzárendelési előzményei.

Oszlop neve	Típus	Leírás
id	BIGINT UNSIGNED PK AI	Egyedi azonosító
vehicle_id	BIGINT UNSIGNED FK NOT NULL	Jármű hivatkozása (vehicles.id)
driver_id	BIGINT UNSIGNED FK NOT NULL	Sofőr hivatkozása (drivers.id)
assigned_from	DATETIME NOT NULL	Hozzárendelés kezdete
assigned_to	DATETIME	Hozzárendelés vége (NULL = aktív)
created_at	DATETIME	Bejegyzés létrehozásának időbélyege

3.3.5 files tábla

A rendszerbe feltöltött fájlok metaadatait tárolja. A tényleges fájlok a szerveren tárolt fájlrendszeren vagy felhőben kerülnek elhelyezésre, a tábla csak a metaadatokat (eredeti fájlnév, tárolt fájlnév, MIME típus, méret, tárolási hely) tartalmazza. Erre hivatkoznak a profilképek, tankolási bizonylatok és szervizszámlák.

Oszlop neve	Típus	Leírás
id	BIGINT UNSIGNED PK AI	Egyedi azonosító
uploaded_by_user_id	BIGINT UNSIGNED FK	Feltöltő felhasználó (users.id)
original_name	VARCHAR(255)	Eredeti fájlnév
stored_name	VARCHAR(255)	Szerveren tárolt fájlnév (egyedi)
mime_type	VARCHAR(120)	Fájl MIME típusa (pl. image/jpeg)
size_bytes	BIGINT UNSIGNED	Fájl mérete bájtban
storage_provider	ENUM('LOCAL','S3')	Tárolási hely
created_at	DATETIME	Feltöltés időbélyege

3.3.6 fuel_logs tábla

A sofőrök üzemanyag-feltöltéseit rögzíti. Minden bejegyzés tartalmazza a tankolás dátumát, a kilométeróra állást, az üzemanyag mennyiségét, a végösszeget, a kút nevét és helyszínét, valamint opcionálisan egy bizonylat fotót. A soft delete (is_deleted) mező lehetővé teszi az adatok megőrzését törlés esetén is.

Oszlop neve	Típus	Leírás
id	BIGINT UNSIGNED PK AI	Egyedi azonosító
vehicle_id	BIGINT UNSIGNED FK	Jármű (vehicles.id)
driver_id	BIGINT UNSIGNED FK	Sofőr (drivers.id)
date	DATETIME NOT NULL	Tankolás dátuma és időpontja
odometer_km	INT	Kilométeróra állás tankoláskor
liters	DECIMAL(10,2) NOT NULL	Feltöltött üzemanyag mennyisége (liter)
total_cost	DECIMAL(10,2) NOT NULL	Tankolás végösszege (Ft)
station_name	VARCHAR(255)	Benzinkút neve (opcionális)
location_text	VARCHAR(255)	Helyszín szöveges leírása (opcionális)
receipt_file_id	BIGINT UNSIGNED FK	Bizonylat fájl hivatkozása (files.id)
is_deleted	TINYINT(1) DEFAULT 0	Soft delete jelző

3.3.7 trips tábla

Az útnyilvántartást tárolja. Minden út rögzíti a kezdő és befejező időpontot, a helyszíneket, a távolságot, valamint a kilométeróra leolvasásokat. A soft delete itt is alkalmazott.

Oszlop neve	Típus	Leírás
id	BIGINT UNSIGNED PK AI	Egyedi azonosító
vehicle_id	BIGINT UNSIGNED FK	Jármű (vehicles.id)
driver_id	BIGINT UNSIGNED FK	Sofőr (drivers.id)
start_time	DATETIME NOT NULL	Út kezdési időpontja
end_time	DATETIME	Út befejezési időpontja
start_location	VARCHAR(255)	Indulási helyszín
end_location	VARCHAR(255)	Érkezési helyszín
distance_km	DECIMAL(10,2)	Megtett távolság (km)
start_odometer_km	INT	Kilométeróra induláskor
end_odometer_km	INT	Kilométeróra érkezéskor
notes	TEXT	Sofőr megjegyzései (opcionális)
is_deleted	TINYINT(1) DEFAULT 0	Soft delete jelző

3.3.8 service_requests tábla

A szervizigények teljes életciklusát nyilvántartja. Az igény a REQUESTED állapotból indul, és az admin jóváhagyja vagy elutasítja. Jóváhagyás után a sofőr feltöltheti a szerviz részleteit, majd az admin lezárja az igényt. Az összes státuszváltás időbélyege és az érintett adminisztrátor adatai naplózásra kerülnek.

Státusz	Leírás
REQUESTED	Sofőr által beadott, elbírálásra váró igény
REJECTED	Admin által elutasított igény
APPROVED	Admin által jóváhagyott, szervizre váró igény
DRIVER_COST	Sofőr feltöltötte a szerviz részleteit és költségét
CLOSED	Admin lezárta az igényt

3.3.9 calendar_events tábla

A naptáresemények táblája mind az adminisztrátorok, mind a sofőrök számára elérhetővé tesz egy naptár funkciót. Az események lehetnek személyes feladatok (PERSONAL_TASK) vagy szerviz időpontok (SERVICE_APPOINTMENT). Az utóbbiak opcionálisan hivatkozhatnak egy konkrét szervizigényre.

3.3.10 notifications tábla

A rendszer különféle eseményekről (jármű hozzárendelés, szerviz státuszváltozás, út vagy tankolás rögzítése) értesítéseket generál a felhasználóknak. Az értesítések típus szerint osztályozottak: ACCOUNT, ASSIGNMENT, FUEL_LOG, SERVICE_REQUEST, TRIP. A is_read mező jelzi, hogy a felhasználó olvasta-e már az értesítést.

3.3.11 password_tokens tábla

A jelszó-visszaállítási folyamathoz szükséges tokeneket tárolja. Minden token egyedi, hash-elt formában kerül mentésre, és rendelkezik egy lejárati időbélyeggel. A felhasználó törlése esetén a tokenek CASCADE DELETE-tel automatikusan törlődnek.

3.4 Adatbázis kapcsolatok

Az alábbi táblázat összefoglalja a legfontosabb idegen kulcs kapcsolatokat és azok viselkedését törlés esetén:

Tábla	Idegen kulcs	Hivatkozott tábla	Törlés hatása
drivers	user_id	users	CASCADE (sofőr törlődik)
vehicle_assignments	vehicle_id	vehicles	RESTRICT (nem törölhető)
vehicle_assignments	driver_id	drivers	RESTRICT (nem törölhető)
fuel_logs	vehicle_id / driver_id	vehicles / drivers	RESTRICT
fuel_logs	receipt_file_id	files	SET NULL
trips	vehicle_id / driver_id	vehicles / drivers	RESTRICT
service_requests	invoice_file_id	files	SET NULL
calendar_events	owner_user_id	users	CASCADE
notifications	user_id	users	CASCADE
password_tokens	user_id	users	CASCADE

4. Backend

4.1 Technológia

A backend ASP.NET Core 10 keretrendszerrel készült, C# programozási nyelven. Az ASP.NET Core egy nyílt forráskódú, platformfüggetlen, nagy teljesítményű webalkalmazás-fejlesztő keretrendszer, amelyet a Microsoft fejleszt. A 10-es verzió a .NET 10 platformra épül, és számos teljesítménybeli és fejlesztői tapasztalatot javító újítást tartalmaz.

Az alkalmazás REST API-t valósít meg, amelyhez a Swagger / OpenAPI dokumentáció is automatikusan generálódik. Fejlesztési módban a Swagger UI elérhető a http://localhost:5000/swagger címen, ami megkönnyíti az API végpontok tesztelését és dokumentálását.

4.2 Projekstruktúra

A backend projekt a clean architecture elveit követi, logikailag elkülönítve a különböző rétegeket. A főbb mappastruktúra:

```
backend/
  Controllers/        ← API végpontok (HTTP kérések fogadása)
  Services/           ← Üzleti logika
  Models/             ← Adatbázis entitások (EF Core)
  DTOs/               ← Adatátviteli objektumok
  appsettings.json    ← Konfiguráció
  Program.cs          ← Alkalmazás belépési pont, DI konfiguráció
```

4.3 Hitelesítés és szerepkörök

A rendszer JWT (JSON Web Token) alapú hitelesítést alkalmaz. Bejelentkezés után a szerver egy aláírt JWT tokent ad vissza, amelyet a kliens minden védett végpont hívásakor az Authorization: Bearer <token> HTTP fejlécben küld el.

A rendszer két felhasználói szerepkört különböztet meg:

- ADMIN – flottaadminisztrátor, hozzáfér az összes adminisztrációs funkcióhoz (sofőrkezelés, járműkezelés, szervizigények jóváhagyása, statisztikák).
- DRIVER – sofőr, csak a saját adatait látja és kezeli (saját utak, tankolások, szervizigények, profil).

A végpontokat [Authorize(Roles = "ADMIN")] illetve [Authorize(Roles = "DRIVER")] attribútumokkal védjük. A mobilos bejelentkezés külön végponton (/api/login-mobile) érhető el, amely kizárólag DRIVER szerepkörű felhasználóknak adja vissza a tokent.

4.4 Jelszótitkosítás

A jelszavakat BCrypt algoritmussal tároljuk az adatbázisban. A BCrypt adaptív hash-elési funkcióval rendelkezik, amelynek köszönhetően a számítási költség beállítható, így a brute force támadásokkal szemben ellenálló marad. A BCrypt tartalmaz egy automatikusan generált salt értéket is, amely a rainbow table alapú támadások ellen nyújt védelmet.

4.5 E-mail funkciók

A backend SMTP protokollon keresztül (Gmail, 587-es port) képes e-maileket küldeni. Az alábbi eseményekhez küld értesítést a rendszer:

Esemény	Értesített	Tartalom
Jelszó-visszaállítás kérés	Érintett felhasználó	Egyedi token link, 60 perces lejárattal
Sofőr profil létrehozása	Sofőr	Jelszó beállítása (egyedi token link, 60 perces lejárattal)
Sofőr profil törlése	Sofőr	Értesítés a törlésről
Sofőr profil aktiválása	Sofőr	Értesítés az aktiválásról

4.6 API végpontok

Az API alap URL-je: https://fleetflow-zarodolgozat-backend-ressdominik.jcloud.jedlik.cloud. Minden védett végpont JWT tokent igényel az Authorization: Bearer fejlécben.

4.6.1 Hitelesítés (/api)

Metódus	Végpont	Szerepkör	Leírás
POST	/api/login	–	Bejelentkezés (web / admin)
POST	/api/login-mobile	–	Bejelentkezés mobilon (csak DRIVER)

4.6.2 Sofőrkezelés (/api/admin/drivers) – ADMIN

Metódus	Végpont	Leírás
GET	/api/admin/drivers	Sofőrök listázása (lapozható, szűrhető)
POST	/api/admin/drivers	Új sofőr létrehozása
PATCH	/api/admin/drivers/edit/{id}	Sofőr adatainak módosítása
PATCH	/api/admin/drivers/activate/{id}	Sofőr aktiválása
PATCH	/api/admin/drivers/deactivate/{id}	Sofőr deaktiválása

4.6.3 Járműkezelés (/api/admin/vehicles) – ADMIN

Metódus	Végpont	Leírás
GET	/api/admin/vehicles	Járművek listázása (lapozható, szűrhető)
POST	/api/admin/vehicles	Új jármű létrehozása
PATCH	/api/admin/vehicles/edit/{id}	Jármű adatainak módosítása
PATCH	/api/admin/vehicles/activate/{id}	Jármű aktiválása
PATCH	/api/admin/vehicles/deactivate/{id}	Jármű deaktiválása (RETIRED)

4.6.4 Jármű–Sofőr hozzárendelés (/api/admin)

Metódus	Végpont	Leírás
GET	/api/admin/assign/driver/{userId}	Sofőr aktív hozzárendelésének lekérése
GET	/api/admin/assign/vehicle/{vehicleId}	Jármű aktív hozzárendelésének lekérése
POST	/api/admin/assign/{userId}/{vehicleId}	Jármű hozzárendelése sofőrhöz
PATCH	/api/admin/unassign/{userId}	Hozzárendelés megszüntetése
GET	/api/admin/assignment/history/{id}	Jármű hozzárendelési előzményei

4.6.5 Útnyilvántartás (/api/trips)

Metódus	Végpont	Szerepkör	Leírás
GET	/api/trips/admin	ADMIN	Összes út listázása (lapozható)
GET	/api/trips/mine	DRIVER	Saját utak listázása (lapozható)
POST	/api/trips	DRIVER	Új út rögzítése
PATCH	/api/trips/delete/{id}	ADMIN, DRIVER	Út soft törlése
PATCH	/api/trips/restore/{id}	ADMIN	Törölt út visszaállítása

4.6.6 Üzemanyag-napló (/api/fuellogs)

Metódus	Végpont	Szerepkör	Leírás
GET	/api/fuellogs/admin	ADMIN	Összes tankolási napló (lapozható)
GET	/api/fuellogs/mine	DRIVER	Saját tankolási naplók (lapozható)
POST	/api/fuellogs	DRIVER	Új tankolás rögzítése
PATCH	/api/fuellogs/delete/{id}	ADMIN, DRIVER	Napló soft törlése
PATCH	/api/fuellogs/restore/{id}	ADMIN	Törölt napló visszaállítása

4.6.7 Szervizigények (/api/service-requests)

Metódus	Végpont	Szerepkör	Leírás
GET	/api/service-requests/admin	ADMIN	Összes szervizigény listázása (lapozható)
GET	/api/service-requests/mine	DRIVER	Saját szervizigények (lapozható)
POST	/api/service-requests	DRIVER	Új szervizigény beadása
DELETE	/api/service-requests/cancel/{id}	DRIVER	Szervizigény visszavonása
PATCH	/api/service-requests/reject/{id}	ADMIN	Szervizigény elutasítása
PATCH	/api/service-requests/approve/{id}	ADMIN	Szervizigény jóváhagyása
PATCH	/api/service-requests/upload-details/{id}	DRIVER	Szerviz részleteinek feltöltése
PATCH	/api/service-requests/edit-uploaded-data/{id}	DRIVER	Szerviz részleteinek módosítása
PATCH	/api/service-requests/close/{id}	ADMIN	Szervizigény lezárása

4.6.8 Naptár (/api/calendarevents)

Metódus	Végpont	Leírás
GET	/api/calendarevents	Naptáresemények listázása
POST	/api/calendarevents	Új naptárelem létrehozása
DELETE	/api/calendarevents/{id}	Naptárelem törlése

4.6.9 Értesítések (/api/notifications)

Metódus	Végpont	Leírás
GET	/api/notifications	Értesítések listázása
POST	/api/notifications	Értesítés küldése
PATCH	/api/notifications/read	Értesítések olvasottnak jelölése
DELETE	/api/notifications/{id}	Értesítés törlése
GET	/api/notifications/unread-status	Van e olvasatlan értesítés

4.6.10 Profil (/api/profile)

Metódus	Végpont	Szerepkör	Leírás
GET	/api/profile/mine	ADMIN, DRIVER	Saját profiladatok
GET	/api/profile/assigned-vehicle	DRIVER	Hozzárendelt jármű adatai
PATCH	/api/profile/edit	ADMIN, DRIVER	Profiladatok módosítása
PATCH	/api/profile/delete-profile-image	ADMIN, DRIVER	Profilkép törlése
POST	/api/profile/forgot-password	–	Jelszóemlékeztető e-mail küldése
POST	/api/profile/set-password	–	Jelszó beállítása tokennel

4.6.11 Statisztikák (/api/statistics)

Metódus	Végpont	Szerepkör	Leírás
GET	/api/statistics/driver/{id}	ADMIN	Adott sofőr statisztikái
GET	/api/statistics/vehicle/{id}	ADMIN	Adott jármű statisztikái
GET	/api/statistics/fuellog	ADMIN	Üzemanyag statisztikák
GET	/api/statistics/trip	ADMIN	Útstatisztikák
GET	/api/statistics/servicerequest	ADMIN	Szerviz statisztikák
GET	/api/statistics/mine	DRIVER	Saját statisztikák
GET	/api/statistics/admin	ADMIN	Admin dashboard összesítő

4.6.12 Fájlkezelés (/api/files)

Metódus	Végpont	Leírás
POST	/api/files	Fájl feltöltése (multipart/form-data)
GET	/api/files/{id}	Fájl letöltése azonosító alapján
DELETE	/api/files/{id}	Fájl törlése
GET	/api/files/thumbnail/{userId}	Felhasználó profilképének lekérése

4.7 Lapozás és szűrés

A lista végpontok egységes lapozási (pagination) és szűrési mechanizmust alkalmaznak. Az összes listázó végpont támogatja a következő query paramétereket:

Paraméter	Típus	Leírás
page	int	Lapszám (alapértelmezett: 1)
pageSize	int	Lapméret (max: 200, alapértelmezett: 25)
stringQ	string	Szabad szöveges keresés (entitástól függően különböző mezőkre)
ordering	string	Rendezési mező neve, _desc utótaggal fordított sorrend

5. Biztonság és rendszerszintű megoldások

5.1 JWT alapú hitelesítés részletei

A JWT (JSON Web Token) egy nyílt szabvány (RFC 7519), amely kompakt, URL-biztos formában teszi lehetővé az információk két fél között való biztonságos átvitelét. A FleetFlow rendszerben a JWT token három részből áll: fejléc (header), tartalom (payload) és aláírás (signature).

A fejlécben az algoritmus típusa (HS256 – HMAC SHA-256) kerül megadásra. A payloadban a felhasználó azonosítója, szerepköre (ADMIN/DRIVER) és a token lejárati ideje van elhelyezve. Az aláírás a szerver titkos kulcsával (Jwt__Key) van generálva, amellyel ellenőrizhető a token hitelessége.

A token lejárata után a kliens automatikusan kijelentkeztetésre kerül, és újra be kell jelentkeznie. A mobil alkalmazásban a SecureStorage gondoskodik a token biztonságos tárolásáról, amely platformonként különböző titkosítású titkos tárolót használ (Android: EncryptedSharedPreferences, iOS: Keychain).

5.2 Middleware réteg

Az ASP.NET Core middleware pipeline az érkező HTTP kérések feldolgozásának sorrendjét határozza meg. A FleetFlow backend a következő middleware-eket alkalmazza:

Middleware	Feladata
UseHttpsRedirection	HTTP kéréseket automatikusan HTTPS-re irányítja át
UseAuthentication	JWT token validálása és a felhasználói identitás beállítása
UseAuthorization	Szerepkör-alapú hozzáférés ellenőrzése az egyes végpontokon
UseSwagger / UseSwaggerUI	API dokumentáció kiszolgálása fejlesztési módban
CORS Middleware	Cross-Origin Resource Sharing engedélyezése a frontend domainek számára
Health Check Middleware	/healthz végpont kiszolgálása Kubernetes probe-okhoz
Exception Handling Middleware	Egységes hibaválaszok generálása belső kivételek esetén

5.3 CORS konfiguráció

A CORS (Cross-Origin Resource Sharing) konfiguráció meghatározza, hogy mely domainek küldhetnek kéréseket a backend API-ra. Fejlesztési módban az összes origintől érkező kérés engedélyezett, produkciós módban csak a konfigurált frontend URL-ek kapnak hozzáférést.

5.4 Adatvalidáció

Az API végpontok bemeneti adatait adatannotációkkal (Data Annotations) validáljuk. A modellek kötelező mezői [Required] attribútummal, a maximális hosszak [MaxLength] attribútummal, az e-mail formátumok [EmailAddress] attribútummal vannak megjelölve. Az érvénytelen kérésekre a szerver automatikusan 400 Bad Request választ ad, a validációs hibák részleteivel együtt.

5.5 Soft delete mechanizmus

Az utak (trips) és tankolási naplók (fuel_logs) nem kerülnek fizikailag törlésre az adatbázisból, hanem egy is_deleted jelzőmező állítódik igaz értékre. Ez biztosítja, hogy az adatok auditnaplózási célokból megőrződjenek, és adminisztrátor által visszaállíthatók legyenek.

A lekérdező végpontok alapértelmezés szerint kizárják a törölt rekordokat, de az ADMIN szerepkörű felhasználók az isDeleted=true query paraméterrel lekérhetik a törölt elemeket is, és a restore végponton visszaállíthatják őket.

5.6 Fájlfeltöltés és -kezelés

A rendszer képes fájlok (profilképek, bizonylatok, számlák) fogadására multipart/form-data formátumban. A SixLabors.ImageSharp könyvtár segítségével a képek automatikusan átméretezésre és optimalizálásra kerülnek feltöltés után, csökkentve a tárhelyigényt és a letöltési időt.

Minden feltöltött fájl metaadatai (eredeti fájlnév, tárolt fájlnév, MIME típus, méret) a files táblában kerülnek nyilvántartásba. A tárolt fájlnév egy egyedi azonosítóval (GUID) van kiegészítve az ütközések elkerülésére. A files.thumbnail/{userId} végponton keresztül a profilképek bélyegkép formátumban is lekérhetők.

6. Backend tesztelés

6.1 Tesztelési megközelítés

A szoftvertesztelés elengedhetetlen része a fejlesztési folyamatnak: segít időben feltárni a hibákat, megerősíti, hogy az implementált funkciók az elvárásoknak megfelelően működnek, és megakadályozza a regressziók megjelenését a kód módosítása során. A FleetFlow backend tesztelése két szinten valósult meg: unit tesztek és API integrációs tesztek formájában.

A backend teszteléséhez xUnit keretrendszert alkalmaztunk, a Moq könyvtárral kombinálva a függőségek mockolásához. A tesztprojekt a tests/ mappában helyezkedik el, és a tests.slnx solution fájlon keresztül hivatkozik a főprojektre.

Az adatbázis teszteléshez Microsoft.EntityFrameworkCore.InMemory csomagot alkalmaztunk, amellyel in-memory adatbázist hozunk létre minden egyes teszt futásakor. Ez biztosítja a tesztek izolációját és függetlenségét egymástól, valamint elkerüli a valós adatbázis igénybevételét. Minden teszt saját, egyedi nevű in-memory adatbázist kap (GUID alapon), így párhuzamosan is futhatnak interferencia nélkül.

6.2 A tesztprojekt struktúrája

A tesztprojekt a következő fájlokból áll:

Fájl / mappa	Leírás
tests/tests.slnx	Solution fájl, amely összeköti a backend és teszt projekteket
tests/tests/tests.csproj	Tesztprojekt (net10.0 target, xUnit + Moq)
tests/tests/Controllers/OwnDataControllerTests.cs	Profil és jelszókezelés unit tesztek
tests/tests/Services/EmailServiceTests.cs	E-mail küldési service unit tesztek
tests/tests/TestHelpers/TestControllerFactory.cs	Controller példányosítás segédosztály
tests/tests/TestHelpers/TestDbContextFactory.cs	In-memory EF Core adatbázis factory
tests/tests/fetch-client-collection_FleetFlow_API_tests.json	API integrációs tesztek (Fetch Client)

6.3 Tesztelési segédosztályok

6.3.1 TestDbContextFactory

Az in-memory adatbázist hozza létre minden tesztesethez. Minden híváskor új, egyedi névvel (GUID) rendelkező adatbázist generál, így a tesztek egymástól teljesen függetlenek. Az EF Core InMemory provider segítségével nem szükséges valós adatbázis-kapcsolat a tesztek futtatásához.

6.3.2 TestControllerFactory

A controllerek példányosítását végzi el a tesztekhez, minden szükséges függőség mockolásával. Az alapértelmezés szerint a következő függőségek vannak mockolva:

- IFileService – fájlkezelési műveletek
- ITokenService – JWT token generálás (fixen "test-token" értéket ad vissza)
- IEmailService – e-mail küldési service (vagy átadott mock, vagy alap mock)

Az Frontend:BaseUrl konfigurációs értéke teszt módban http://localhost:5174 értékre van beállítva.

6.4 Unit tesztek

6.4.1 OwnDataControllerTests

Ez a tesztosztály az OwnDataController (profil és jelszó kezelés) funkcionalitását teszteli:

- GetOwnProfile_ShouldReturnUnauthorized_WhenNoUser – Ha nincs felhasználó a request kontextusban, a végpont 401 Unauthorized választ ad vissza. A teszt ellenőrzi, hogy a controller megfelelően kezeli a hitelesítés nélküli kéréseket.
- ForgotPassword_ShouldSendEmail_WhenUserExists – In-memory adatbázisba szúr be egy aktív felhasználót, majd meghívja a ForgotPassword() metódust. Ellenőrzi, hogy az IEmailService.SendAsync() pontosan egyszer hívódott meg, és a megfelelő e-mail kerül kiküldésre.

6.4.2 EmailServiceTests

Az e-mail küldési funkcionalitást teszteli az EmailService osztályon:

- SendAsync_ShouldNotThrow – In-memory konfigurációval (SMTP host, port, feladó, megjelenő név) hozza létre az EmailService-t, majd meghívja a SendAsync() metódust valós paraméterekkel. A teszt célja, hogy a hívás ne dobjon kivételt. A teszt AddUserSecrets<EmailServiceTests>()-t is alkalmaz, így fejlesztői gépen a user secrets-ből is felvehetők a hiányzó SMTP hitelesítési adatok.

6.5 API integrációs tesztek (Fetch Client kollekció)

Az API végpontok valós működésének tesztelésére egy Fetch Client teszt kollekciót hoztunk létre (fetch-client-collection_FleetFlow_API_tests.json). Ez a kollekció az élő backend URL-t (Jedlik Cloud) célozza, és Bearer tokennel hitelesített kéréseket küld.

#	Teszt neve	HTTP metódus	Végpont	Leírás
1	Login	POST	/api/login	Bejelentkezés helyes e-mail/jelszó párral
2	Get Own Profile	GET	/api/profile/mine	Saját profil lekérése JWT tokennel
3	Edit Service Details	PATCH	/api/service-requests/edit-uploaded-data/1	Szerviz adatok módosítása
4	Delete Notification	DELETE	/api/notifications/2	Értesítés törlése azonosító alapján
5	Create Fuel Log	POST	/api/fuellogs	Új tankolási bejegyzés rögzítése

6.6 CI/CD integrált tesztelés

A GitHub Actions CI/CD pipeline automatikusan futtatja az összes unit tesztet minden main branch-re kerülő push esetén. Két workflow gondoskodik erről:

- dotnet.yml – A .NET CI workflow elvégzi a build és a tesztek futtatását minden push esetén.
- tests.yml – Dedikált tesztelési workflow, amely csak a teszt lépéseket futtatja le.

Ha bármelyik teszt megbukik, a deploy folyamat leáll, és az új verzió nem kerül ki a Kubernetes klaszterre. Ez biztosítja, hogy a produkciós környezetbe csak tesztelten stabil kód kerülhessen.

A tesztek futtatásához az alábbi parancs használható:

```
dotnet test tests/tests.slnx
```

7. Konténerizáció és publikálás

7.1 Docker

A backend alkalmazás Docker konténerben fut, amelyet a GitHub Container Registry (GHCR) tárol. A Dockerfile multi-stage build stratégiát alkalmaz a minimális méretű végső image érdekében:

Build szakasz	Alap image	Leírás
base	mcr.microsoft.com/dotnet/aspnet:10.0	Futtatási környezet (8080-as port)
build	mcr.microsoft.com/dotnet/sdk:10.0	Fordítási szakasz
publish	build	Release publikálás optimalizálással
final	base	Végső, minimális image

A Docker image automatikusan buildelődik és töltődik fel a CI/CD pipeline által a GHCR-be:

```
ghcr.io/fleetflow-zarodolgozat/fleetflow-backend:latest
```

7.2 Kubernetes

Az alkalmazás Kubernetes orchestráción fut a Jedlik Cloud infrastruktúrán. A deployment.yaml konfigurációs fájl definiálja az összes szükséges Kubernetes erőforrást.

Erőforrás neve	Típus	Leírás
fleetflow-api-deployment	Deployment	Az API konténer futtatása, Rolling Update stratégiával
fleetflow-api-cluster-ip-service	Service (ClusterIP)	Belső hálózati elérés (port 80 → 8080)
fleetflow-api-ingress	Ingress (nginx)	Külső HTTPS elérés, SSL terminálás

A live backend URL:

```
https://fleetflow-zarodolgozat-backend-ressdominik.jcloud.jedlik.cloud
```

A Kubernetes deployment health probe-okkal van konfigurálva, amelyek biztosítják, hogy csak egészséges pod kapjon forgalmat:

Probe típusa	Útvonal	Konfiguráció
startupProbe	GET /healthz	Max. 30 × 2 másodperc várakozás indulásra
readinessProbe	GET /healthz	10 másodpercenként ellenőriz; nem kész pod nem kap forgalmat
livenessProbe	GET /healthz	20 másodpercenként ellenőriz; nem válaszoló pod újraindul

7.3 CI/CD Pipeline

A GitHub Actions három workflow fájlt tartalmaz, amelyek a fejlesztési folyamat különböző szakaszait automatizálják:

7.3.1 .NET CI (dotnet.yml)

Minden main branch push-ra fut. Elvégzi a kód checkout-ját, a .NET 10 SDK telepítését, a NuGet csomagok visszaállítását, a fordítást (build), és a tesztek futtatását.

7.3.2 Docker Build és Deploy (deploy.yml)

Minden main branch push-ra fut a .NET CI sikeres futása után. A következő lépések zajlanak le: Docker Buildx előkészítése, bejelentkezés a GHCR-be, Docker image buildelése és feltöltése a GHCR-be, kubectl telepítése, Kubernetes konfiguráció beállítása, és végül a deployment alkalmazása a Kubernetes klaszterre.

7.3.3 Android Build (android-build.yml – Mobil)

A mobilalkalmazás repositoryjában elhelyezett workflow automatikusan buildeli az Android APK-t és létrehozza a GitHub Release-t minden tag push esetén.

7.4 Fejlesztői futtatási kézikönyv

A projekt lokális futtatásához az alábbi előfeltételek szükségesek:

- .NET 10 SDK (https://dotnet.microsoft.com/download)
- MySQL 8+ szerver
- Docker Desktop (opcionális, Dockeres futtatáshoz)

A backend lokális indítása lépései:

```
# 1. Klónozd a repót
git clone https://github.com/FleetFlow-Zarodolgozat/FleetFlow_backend.git
cd FleetFlow_backend
```

```
# 2. Adatbázis inicializálása
mysql -u root -p < create_tables.sql
mysql -u root -p < add_datas.sql   # opcionális mintaadatok
```

```
# 3. Titkos beállítások konfigurálása
cd backend/backend
dotnet user-secrets set "ConnectionStrings:DefaultConnection"
"server=localhost;database=flottakezelo_db;user=root;password=jelszo"
dotnet user-secrets set "Jwt:Key" "min-32-karakteres-jwt-kulcs-ide"
dotnet user-secrets set "EmailSettings:Username" "email@gmail.com"
dotnet user-secrets set "EmailSettings:Password" "alkalmazas-jelszo"
```

```
# 4. Alkalmazás indítása
dotnet run
```

Az alkalmazás fejlesztési módban elérhető a http://localhost:5000 címen, a Swagger UI a http://localhost:5000/swagger oldalon.

7.5 Környezeti változók

Az alkalmazás konfigurálásához a következő környezeti változókat kell beállítani (Docker/Kubernetes esetén ezek a deployment.yaml-ben kerülnek megadásra, lokális fejlesztéshez dotnet user-secrets-szel):

Változó neve	Leírás	Példaérték
ConnectionStrings__DefaultConnection	MySQL kapcsolati string	server=db;database=flottakezelo_db;user=root;password=...
Jwt__Key	JWT aláíró kulcs (min. 32 karakter)	super-secret-key-min-32-chars-here
Jwt__Issuer	JWT kibocsátó azonosítója	FleetFlow app
Jwt__Audience	JWT célközönség	FleetFlowUsers
EmailSettings__From	Feladó e-mail cím	fleetflow.info@gmail.com
EmailSettings__DisplayName	Megjelenő feladónév	FleetFlow
EmailSettings__SmtpHost	SMTP szerver hostname	smtp.gmail.com
EmailSettings__SmtpPort	SMTP port száma	587
EmailSettings__Username	SMTP hitelesítési felhasználónév	your@gmail.com
EmailSettings__Password	SMTP App jelszó	your-app-password
Frontend__BaseUrl	Frontend alap URL (jelszó-visszaállítás linkhez)	https://yourfrontend.example.com
ASPNETCORE_ENVIRONMENT	Futtatási környezet	Production / Development

7.6 GitHub Actions Secrets

A CI/CD pipeline a következő GitHub Secrets értékeket igényli a deploy workflow futtatásához:

Secret neve	Leírás
DB_CONNECTION_STRING	MySQL kapcsolati string a produkciós adatbázishoz
JWT_KEY	JWT aláíró kulcs (ugyanaz mint a Jwt__Key)
EMAIL_USERNAME	SMTP felhasználónév
EMAIL_PASSWORD	SMTP App jelszó
FRONTEND_BASE_URL	A produkciós frontend URL-je
GHCR_PAT	GitHub Personal Access Token (packages:write jogosultsággal)
KUBE_CONFIG_DATA	Base64-kódolt Kubernetes kubeconfig fájl

8. Mobil alkalmazás

8.1 Technológia és architektúra

A mobil alkalmazás .NET MAUI (Multi-platform App UI) keretrendszerrel készült C# és XAML technológiák felhasználásával. A .NET MAUI egyetlen kódbázisból teszi lehetővé natív Android, iOS, macOS Catalyst és Windows alkalmazások fejlesztését.

Az alkalmazás kizárólag sofőrök (DRIVER szerepkörű felhasználók) számára készült. Adminisztrációs funkciókat nem tartalmaz – azok a webes felületen érhetők el.

Technológia	Verzió	Cél
.NET MAUI	net10.0	Többplatformos UI keretrendszer
C#	12+	Alkalmazáslogika
XAML	–	UI elrendezés definíció
CommunityToolkit.Maui	14.0.1	Felugrók, konverterek, UI kiterjesztések
CommunityToolkit.Mvvm	8.4.0	MVVM forrásgenerátorok, parancsok
Plugin.Maui.Calendar	3.0.1	Interaktív naptár komponens
SkiaSharp	3.119.2	2D grafikai megjelenítés
Microsoft.Extensions.Http	10.0.4	Típusos HttpClient DI
FontAwesome	–	Ikon betűkészletek

Támogatott platformok: Android (API 21+), iOS (15.0+), macOS Catalyst (15.0+), Windows 10.

8.2 Architektúra – MVVM

Az alkalmazás az MVVM (Model-View-ViewModel) tervezési mintát követi a CommunityToolkit.Mvvm forrásgenerátorainak segítségével. Ez a minta tisztán elválasztja az üzleti logikát a megjelenítéstől:

- Modellek – Egyszerű C# osztályok, amelyek az API válaszait reprezentálják (Driver, Trip, Fuel, Service, stb.)
- Services – Típusos HttpClient burkolók, amelyek a REST API-val kommunikálnak.
- ViewModelek – Adatokat és parancsokat tesznek elérhetővé a UI számára. ObservableObject alaposztályból és [RelayCommand] attribútumokból építkeznek.
- Nézetek (Pages/Popups) – XAML alapú UI, amely adatkötéssel kapcsolódik a ViewModelekhez.
- Dependency Injection – Minden Service, ViewModel és Page regisztrálva van a MauiProgram.cs fájlban.

8.3 Hitelesítés és munkamenet-kezelés

A hitelesítési folyamat a következő lépésekből áll:

- A sofőr megadja e-mail címét és jelszavát a Bejelentkezés képernyőn.
- Az AuthService elküldi a POST /api/login-mobile kérést.
- Siker esetén a szerver visszaad egy JWT Bearer tokent.
- A SessionService.SaveToken() biztonságosan elmenti a tokent a .NET MAUI SecureStorage-ba.
- Az AuthHttpHandler (DelegatingHandler) automatikusan hozzáfűzi az Authorization: Bearer fejlécet minden kimenő HTTP kéréshez.
- Kijelentkezéskor a SessionService.Logout() eltávolítja a tokent és az app visszanavigál a bejelentkezési képernyőre.

8.4 Alkalmazás képernyői

8.4.1 Bejelentkezés képernyő

Az alkalmazás indulásakor megjelenő képernyő. A sofőrök e-mail és jelszó megadásával jelentkeznek be. A képernyő tartalmaz jelszó megjelenítés/elrejtés kapcsolót, „Elfelejtett jelszó?" gombot, és betöltésjelzőt bejelentkezés közben. A mobilos bejelentkezés a /api/login-mobile végpontot hívja, amely csak DRIVER szerepkörű felhasználóknak ad tokent.

8.4.2 Irányítópult (Dashboard)

A bejelentkezés utáni fő képernyő. Teljes áttekintést nyújt a sofőr munkájáról: megjelenik a sofőr profilkártyája, a hozzárendelt jármű adatai, 6 statisztikai kártya (utak száma, össztávolság, szervizdíjak, tankolások) és egy interaktív naptár, amely a napi eseményeket (utak, szervizek) pontokkal jelzi. A csengő ikon az olvasatlan értesítések számát mutatja.

8.4.3 Utak képernyő (Trip Log)

A sofőrök itt rögzíthetik, tekinthetik meg és törölhetik útjaikat. Az útfelvétel kinyitható űrlapot tartalmaz a következő mezőkkel: kezdő/befejező dátum és idő, indulási és érkezési helyszín, távolság (km), kezdő/záró kilométeróra, megjegyzések. A lista lapozható, 10 elemet jelenít meg oldalanként. Törlés előtt megerősítő párbeszédablak jelenik meg.

8.4.4 Tankolás képernyő (Fuel Log)

Az üzemanyag-naplók kezelése. Az új tankolás felvételénél a kötelező mezők (dátum, liter, összköltség) mellett opcionálisan megadható a kút neve, a helyszín és egy bizonylat fotó (galériából vagy kamerával). A fotó multipart/form-data formátumban kerül feltöltésre az API felé. A lista lapozható.

8.4.5 Szerviz képernyő (Service Requests)

A sofőrök szervizigényeket adhatnak be (cím + leírás), tekinthetik meg korábbi igényeiket és azok státuszát (REQUESTED, REJECTED, APPROVED, DRIVER_COST, CLOSED). A „Details" gombra nyomva egy felugró ablakban tölthetők fel a szerviz részletei: a sofőr által jelentett költség (Ft), lezárási megjegyzés és számla fotó. A státuszok színkódolással jelennek meg.

8.4.6 Értesítések képernyő

Az összes rendszerértesítés listája. Az értesítések típus szerint ikon-nal és színkódolással (ACCOUNT, ASSIGNMENT, FUEL_LOG, SERVICE_REQUEST, TRIP) jelennek meg. Az olvasatlan értesítések kék ponttal vannak jelölve. Az összes egyszerre olvasottnak jelölhető, és egyenként törölhetők.

8.4.7 Profil képernyő

A sofőr személyes adatait (név, e-mail, telefon, jogosítvány adatok) jeleníti meg, és lehetővé teszi azok szerkesztését. Profil- és jelszócsere is elvégezhető. Profilkép galériából vagy kamerával tölthető fel, vagy törölhető. A témabeállítás (Rendszer / Világos / Sötét) is itt érhető el, és Preferences-ben tárolódik. A kijelentkezés gomb a bejelentkezési képernyőre navigál vissza.

8.5 Navigáció

Az alkalmazásban az oldalak közti navigáció az AppShell.xaml-ben definiált útvonalakon keresztül történik (Shell.Current.GoToAsync()). Az alsó navigációs sáv (BottomNavigation komponens) minden fő képernyőn megjelenik, és kiemeli az aktuálisan aktív fület.

Útvonal	Oldal	Leírás
LoginPage	LoginPage	Hitelesítés (induló oldal)
DashboardPage	DashboardPage	Fő kezdőképernyő
TripPage	TripPage	Útnyilvántartás kezelése
FuelPage	FuelPage	Tankolás napló kezelése
ServicePage	ServicePage	Szervizigény kezelés
NotificationPage	NotificationPage	Értesítések listája
ProfilePage	ProfilePage	Profil és beállítások

8.6 Felugró ablakok (Popups)

A mobilalkalmazás három CommunityToolkit.Maui alapú felugró ablakot alkalmaz, amelyek a főoldalakon belül jelennek meg anélkül, hogy új képernyőre kellene navigálni.

8.6.1 Elfelejtett jelszó felugró (ForgotPasswordPopup)

A Bejelentkezés képernyőről nyílik meg az „Elfelejtett jelszó?" gombra kattintva. Tartalmaz egy e-mail beviteli mezőt, „Küldés" gombot (POST /api/profile/forgot-password végpont hívásával) és „Mégse" gombot. Betöltésjelzőt és hibaüzenetet is megjelenít, ha az e-mail cím nem található a rendszerben.

8.6.2 Szerviz részletek felugró (ServiceDetailsPopup)

A Szerviz képernyőn egy szervizigény kártyájának „Details" gombjára kattintva nyílik meg. Az admin által jóváhagyott szervizigények esetén a sofőr itt tölthet fel részleteket: a szerviz valós költségét (Ft-ban), lezárási megjegyzést és számla fotót (galériából vagy kamerával). Az adatok PATCH kéréssel kerülnek az API-ra, a számla fotó multipart/form-data formátumban.

8.6.3 Naptár nap felugró (CalendarDayPopup)

Az Irányítópult interaktív naptárán egy napra kattintva nyílik meg. Megjeleníti az adott napon rögzített összes eseményt (utakat és szerviz időpontokat) az esemény típusával, címével és kezdő/záró időpontjával együtt. Lehetőséget ad új naptárelem rögzítésére is az adott napra. Bezáráskor a naptár automatikusan frissül.

8.7 Adatmodellek

A mobilalkalmazás adatmodelljei a backend API válaszait tükrözik. Minden modell egyszerű C# POCO (Plain Old CLR Object) osztály, amelyeket a JSON deszerializáláshoz használunk.

Modell	Főbb tulajdonságok	Kapcsolódó API végpont
Driver	Id, FullName, Email, Phone, LicenseNumber, LicenseExpiryDate	/api/profile/mine
Vehicle	BrandModel, LicensePlate, Year, CurrentMileageKm, Vin, Status	/api/profile/assigned-vehicle
Stats	TotalTrips, TotalDistance, TotalServices, TotalServicesCost, TotalFuels, TotalFuelCost	/api/statistics/mine
Trip	Id, StartTime, StartLocation, EndLocation, DistanceKm, Notes, LicensePlate	/api/trips/mine
Fuel	Id, Date, Liters, TotalCostCur, StationName, ReceiptFileId, LicensePlate	/api/fuellogs/mine
Service	Id, Title, Description, Status, ScheduledStart, DriverReportCost, LicensePlate	/api/service-requests/mine
Notification	Id, Type, Title, Message, IsRead, RelatedServiceRequestId, CreatedAt	/api/notifications
CalendarEvent	Id, Title, EventType, StartAt, EndAt, RelatedServiceRequestId	/api/calendarevents
PagedResponse<T>	Items, TotalCount, Page, PageSize, TotalPages	Minden listázó végpont

8.8 Téma támogatás

Az alkalmazás három megjelenési módot támogat, amelyek a Profil képernyőn választhatók:

Mód	Viselkedés	Tárolás
Rendszer (System)	Követi az eszköz rendszer szintű beállítását	Preferences ("theme" kulcs)
Világos (Light)	Mindig világos témát használ	Preferences ("theme" kulcs)
Sötét (Dark)	Mindig sötét témát használ	Preferences ("theme" kulcs)

A ThemeService felel a téma alkalmazásáért és tárolásáért. Az Application.Current.UserAppTheme tulajdonság beállításával azonnal érvénybe lép a változtatás. Minden képernyő és komponens AppThemeBinding XAML kiterjesztéssel reagál a témaváltásra, így külön kód nélkül alkalmazkodik a kiválasztott stílushoz.

8.9 CI/CD és kiadások

A mobil repository GitHub Actions workflow-val rendelkezik, amely automatikusan buildeli az Android APK-t és tesz közzé GitHub Release-t minden verzió tag-re (pl. v1.1). Az eddig kiadott verziók:

Verzió	Megjelenés dátuma	Megjegyzés
v1.0	2026. 03. 16.	Első publikus kiadás
v1.0.1	2026. 03. 17.	Minimális UI változtatás
v1.0.2	2026. 03. 18.	Minimális UI változtatás
v1.0.3	2026. 03. 18.	Szerviz díj feltöltésének javítása
v1.0.4	2026. 03. 20.	Minimális UI változtatás
v1.1	2026. 03. 22.	Hibajavítások és UI fejlesztések
v1.2	2026. 04. 07.	Minimális UI változtatás, WIFI hibakezelés
v1.3	2026. 04. 19.	Latest - UI hibák kijavítása, új adatok megjelenítése (TripPage)

9. Webes frontend

9.1 Technológia

A webes frontend React 19 és Vite 7 technológiákkal készült, JavaScript nyelven. A Vite egy rendkívül gyors, modern frontend build eszköz, amely fejlesztési módban azonnali HMR (Hot Module Replacement) frissítést biztosít, production módban pedig optimalizált, minifikált bundle-t generál. Az alkalmazás React Router DOM 7-et használ az oldalak közötti navigációhoz, Bootstrap 5-öt és React-Bootstrap-ot a reszponzív UI komponensekhez. Az oldal URL-je: https://fleetflow.jcloud.jedlik.cloud

Technológia	Verzió	Cél
React	19	Komponens alapú UI könyvtár
Vite	7	Gyors build eszköz és fejlesztői szerver
React Router DOM	7	Kliensoldali routing és navigáció
Bootstrap / React-Bootstrap	5	Reszponzív UI komponensek és stílusok
Axios	1	HTTP kommunikáció a FleetFlow backend API-val
Leaflet / React-Leaflet	1.9 / 5	Interaktív térkép az útvonalakhoz
React Big Calendar	1	Naptár nézet naptáreseményekhez
Cypress	15	End-to-end (E2E) automatikus tesztelés
Nginx	alpine	Statikus fájlok kiszolgálása konténerben
Node.js	20 LTS	Build és fejlesztői környezet

9.2 Projektstruktúra

A frontend projekt a következő mappaszerkezetben épül fel:

```
FleetFlow_frontend/
├── Frontend/
│   ├── src/
│   │   ├── components/    # Újrahasználható komponensek (Sidebar, Footer,
RouteMap)
│   │   ├── pages/         # Oldalak (Login, Dashboard, Vehicles, Drivers, stb.)
│   │   ├── contexts/      # React Context (pl. AuthContext)
│   │   ├── services/      # API hívások (Axios service-ek)
│   │   └── styles/        # CSS stíluslapok
│   ├── cypress/           # E2E Cypress tesztek
│   ├── Dockerfile         # Multi-stage Docker build
│   ├── deployment.yaml    # Kubernetes manifesztek
│   ├── nginx.conf         # Nginx konfiguráció (proxy, gzip, cache)
│   └── package.json
└── .github/workflows/     # CI/CD workflow fájlok
```

9.3 Hitelesítés és AuthContext

Az alkalmazás JWT alapú hitelesítést alkalmaz, amelyet az AuthContext kezeli. Bejelentkezéskor a szerver által visszaküldött JWT Bearer tokent a kliens a sessionStorage-ban tárolja, és Axios interceptor segítségével automatikusan hozzáadja minden kimenő API kérés Authorization fejlécéhez.

A React Context API (AuthContext) gondoskodik arról, hogy a bejelentkezett felhasználó adatai (azonosító, szerepkör) az egész alkalmazásban elérhetők legyenek. A védett útvonalak (protected routes) ellenőrzik, hogy a felhasználó be van-e jelentkezve, és ha nem, a bejelentkezési oldalra irányítják.

9.4 Oldalak és funkciók

9.4.1 Bejelentkezés oldal

Az alkalmazás indulásakor megjelenő oldal. Az adminisztrátorok és sofőrök egyaránt e-mail és jelszó megadásával jelentkeznek be. Sikeres bejelentkezés után a felhasználó szerepköre (ADMIN vagy DRIVER) határozza meg, melyik irányítópultra kerül átirányításra.

9.4.2 Admin irányítópult (Dashboard)

Az adminisztrátori kezdőképernyő összesített statisztikákat jelenít meg a flotta aktuális állapotáról: aktív sofőrök és járművek száma, nyitott szervizigények, legutóbbi utak és tankolások. Az oldal React Big Calendar komponenssel naptárnézetet is tartalmaz, amelyen az összes ütemezett esemény (szerviz időpontok, feladatok) megjelenik.

9.4.3 Járműkezelés

Táblázatos nézetben listázza a flotta összes járművét szűrési és keresési lehetőségekkel (rendszám, márka/modell, státusz). Az adminisztrátor itt adhat hozzá új járművet, szerkesztheti a meglévők adatait, változtathatja a státuszukat (ACTIVE / RETIRED), és kezelheti a jármű–sofőr hozzárendeléseket. A hozzárendelési előzmények is megtekinthetők.

9.4.4 Sofőrkezelés

Az összes sofőr listázása kereshető, szűrhető táblázatban (név, e-mail, jogosítvány lejárata, aktív/inaktív állapot). Új sofőr felvétele, meglévő adatainak módosítása, aktiválása és deaktiválása végezhető el ezen az oldalon. A sofőr profiljához tartozó statisztikák (összes út, távolság, szervizek) is megtekinthetők.

9.4.5 Útnyilvántartás

Admin nézetben az összes sofőr összes útja látható lapozható, szűrhető táblázatban. Az utak részletes adatai (indulási és érkezési helyszín, távolság, kilométeróra adatok, időtartam) megtekinthetők. A Leaflet / React-Leaflet alapú térképes nézet (RouteMap komponens) vizuálisan jeleníti meg az útvonalakat interaktív térképen.

9.4.6 Üzemanyag-naplók

Az összes tankolási bejegyzés megtekintése lapozható táblázatban, sofőr és időszak szerinti szűréssel. Az admin itt tekintheti meg a feltöltött bizonylat fotókat, és szükség esetén visszaállíthatja a soft delete-tel törölt bejegyzéseket. Statisztikai összesítők is megjelennek az üzemanyag-felhasználásról.

9.4.7 Szervizigények kezelése

Az összes beérkező szervizigény listázása státusz szerinti szűréssel (REQUESTED, REJECTED, APPROVED, DRIVER_COST, CLOSED). Az adminisztrátor egy kártyán belül jóváhagyhatja vagy elutasíthatja az igényt, megtekintheti a sofőr által feltöltött szerviz részleteket (valós költség, számla fotó, lezárási megjegyzés), és lezárhatja a szervizfolyamatot.

9.4.8 Sofőr irányítópult

A DRIVER szerepkörű felhasználók egy egyszerűsített irányítópultot látnak, amely a hozzárendelt jármű adatait, a legutóbbi útjaikat és tankolásaikat, valamint a nyitott szervizigényeiket mutatja. Gyors gombok segítségével közvetlenül elindítható az új út, tankolás vagy szervizigény rögzítése.

9.4.9 Sofőr saját oldalak

A sofőrök saját útjaikat, tankolásaikat és szervizigényeiket kezelhetik az erre dedikált oldalakon. Új bejegyzések rögzítése, meglévők törlése (soft delete), és a szervizigények részleteinek feltöltése mind elérhető. A profil oldalon módosítható a jelszó és a profilkép.

9.4.10 Értesítések

Minden felhasználónak (admin és sofőr egyaránt) elérhető egy értesítési oldal, amely listázza a rendszer által generált értesítéseket. Az értesítések típus szerint (ACCOUNT, ASSIGNMENT, FUEL_LOG, SERVICE_REQUEST, TRIP) vizuálisan elkülönülnek. Az olvasatlan értesítések kiemelten jelennek meg, és egyszerre az összes olvasottnak jelölhető.

9.5 Reszponzivitás

Az alkalmazás Bootstrap 5 alapú reszponzív layoutot alkalmaz, amely asztali és mobil böngészőkön egyaránt megfelelően jelenik meg. A Bootstrap grid rendszer és a flexbox alapú komponensek automatikusan alkalmazkodnak a különböző képernyőméretekhez. A Sidebar navigáció kisebb képernyőkön összecsukható drawer formában jelenik meg.

9.6 Webes tesztelés – Cypress E2E

A webes frontend automatizált end-to-end (E2E) tesztelésére Cypress 15 keretrendszert alkalmaztunk. A Cypress tesztek valódi böngészőben futnak, és szimulálják a felhasználói interakciókat (kattintások, űrlapkitöltés, navigáció), ellenőrizve, hogy az alkalmazás az elvárásoknak megfelelően működik.

A Cypress tesztek a Frontend/cypress/ mappában találhatók. A tesztek a CI/CD pipeline Node.js workflow-jában automatikusan futnak minden main branch push esetén, biztosítva, hogy a kulcsfunkciók minden deploy előtt ellenőrzöttek legyenek.

A tesztek futtatása lokálisan:

```
cd FleetFlow_frontend/Frontend
npm run cypress:run     # Headless futtatás
npm run cypress:open    # Interaktív Cypress UI
```

9.7 Docker és Kubernetes

A frontend kétlépéses (multi-stage) Docker build stratégiát alkalmaz. Az első szakaszban a Node.js 20 Alpine image buildeli az alkalmazást (npm ci + npm run build), a második szakaszban az nginx:alpine image szolgálja ki a generált statikus fájlokat. Ez minimális méretű végső image-et eredményez.

Build szakasz	Alap image	Feladat
builder	node:20-alpine	Függőségek telepítése, production build generálása
final	nginx:alpine	Statikus dist/ fájlok kiszolgálása 80-as porton

Az Nginx konfiguráció tartalmaz API proxy beállítást (/api útvonal → backend service), gzip tömörítést a statikus fájlokhoz, megfelelő cache fejléceket és biztonsági HTTP fejléceket.

Kubernetes deploymentje a Frontend/deployment.yaml fájlban van definiálva, amelyben RollingUpdate stratégia biztosítja a leállás nélküli frissítést (maxSurge: 1, maxUnavailable: 0). A cert-manager automatikusan kezeli a Let's Encrypt TLS tanúsítványt a HTTPS eléréshez.

Erőforrás	Típus	Leírás
fleetflow-frontend	Deployment	1 replika, RollingUpdate stratégia
fleetflow-frontend-svc	Service (ClusterIP)	80-as port kiszolgálása
fleetflow-frontend-cert	Certificate	Let's Encrypt TLS tanúsítvány (cert-manager)
fleetflow-frontend-ingress	Ingress (nginx)	HTTPS redirect, külső elérés

9.8 CI/CD pipeline

A frontend repository két GitHub Actions workflow-t tartalmaz:

Node.js CI (node.js.yml) – Minden main ágra kerülő push és pull request esetén fut: Node.js 20 telepítése, npm ci, npm run build (production build), valamint Cypress E2E tesztek futtatása. Ha bármelyik lépés meghiúsul, a deploy nem indul el.

Frontend Deploy (deploy.yml) – Kizárólag main ágra kerülő push esetén fut, a CI sikeres teljesítése után: Docker image buildelése és pusholása a GHCR-be, régi tag nélküli image-ek törlése (max. 5 verzió megtartásával), majd kubectl apply a frissített deployment alkalmazásához a Kubernetes klaszterre.

9.9 Fejlesztői futtatási kézikönyv

Előfeltételek: Node.js 20+, npm. A lokális futtatás lépései:

```
# 1. Repository klónozása
git clone https://github.com/FleetFlow-Zarodolgozat/FleetFlow_frontend.git
cd FleetFlow_frontend/Frontend
# 2. Függőségek telepítése
npm ci
```

```
# 3. Környezeti változó beállítása (vite.config.js proxy szükséges)
# Backend URL alapból: http://localhost:5000
```

```
# 4. Fejlesztői szerver indítása
npm run dev   # http://localhost:5174
```

```
# Production build
npm run build
```

```
# Linter futtatása
npm run lint
```

A fejlesztői szerver alapértelmezés szerint a http://localhost:5174 címen érhető el. A Vite konfigurációban API proxy van beállítva, amely az /api útvonalú kéréseket a backend szerverre irányítja.

10. Források / Irodalomjegyzék

- Microsoft .NET Documentation: https://docs.microsoft.com/dotnet (2026. 04. 07.)
- ASP.NET Core Documentation: https://docs.microsoft.com/aspnet/core (2026. 04. 07.)
- Entity Framework Core: https://docs.microsoft.com/ef/core (2026. 04. 07.)
- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/ (2026. 04. 07.)
- .NET MAUI Documentation: https://docs.microsoft.com/dotnet/maui(2026. 04. 07.)
- CommunityToolkit.Mvvm: https://learn.microsoft.com/dotnet/communitytoolkit/mvvm/ (2026. 04. 07.)
- JWT.io – JSON Web Tokens: https://jwt.io/ (2026. 04. 07.)
- BCrypt.Net Documentation: https://github.com/BcryptNet/bcrypt.net (2026. 04. 07.)
- Docker Documentation: https://docs.docker.com/ (2026. 04. 07.)
- Kubernetes Documentation: https://kubernetes.io/docs/ (2026. 04. 07.)
- GitHub Actions Documentation: https://docs.github.com/actions (2026. 04. 07.)
- Swagger / OpenAPI Initiative: https://swagger.io/ (2026. 04. 07.)
- xUnit Testing Framework: https://xunit.net/ (2026. 04. 07.)
- Moq Mocking Library: https://github.com/moq/moq4 (2026. 04. 07.)
- React Documentation: https://react.dev/ (2026. 04. 07.)
- Cypress Documentation: https://docs.cypress.io/ (2026. 04. 07.)
- Wikipedia – Kubernetes: https://hu.wikipedia.org/wiki/Kubernetes (2026. 04. 07.)
- Wikipedia – GitHub: https://hu.wikipedia.org/wiki/GitHub (2026. 04. 07.)
- Wikipedia – MySQL: https://hu.wikipedia.org/wiki/MySQL (2026. 04. 07.)

Mellékletek

A mellékelt mappában megtalálható az adatbázis diagram, a projekt logója, valamint a UI tervek.
Mappa elérése lokálisan: Mappa_megnyitása
Mappa letöltése: Letöltés